

Explications become ansible.cfg

```
driss@master-06: /projets_optimedit/iis_infras/infra_iis_cert_wc_ansible_v2.14.18
GNU nano 7.2
[defaults]
# Chemin vers ton inventaire par défaut
inventory          = ./inventory.yml

# Chemin vers tes rôles personnalisés
roles_path         = ./roles

# STRATÉGIE DE RECHERCHE DES COLLECTIONS :
# 1. ./collections : Priorité au répertoire local du projet
# 2. /usr/share/ansible/collections : Fallback au système
collections_paths = ./collections:/usr/share/ansible/collections

# Désactivation du check SSH pour gagner du temps et éviter les erreurs interactives
host_key_checking = False

# Gestion des redirections (pour éviter les warnings de redirection vers ansible.windows)
interpreter_python = auto_silent

[privilege_escalation]
# Configuration pour l'exécution Windows
become = False
become_method = runas
become_user = admin_ansible
```

Ces paramètres contrôlent le **privilège d'escalade** (le passage de droits) lors de des connexions vers les serveurs Windows. Voici l'explication précise :

1. become = False

- **Signification** : Cela signifie que, par défaut, Ansible ne cherche pas à changer d'identité ou à élever ses privilèges une fois connecté au serveur distant.
- **Pourquoi False dans ton cas ?** Dans les environnements Windows via WinRM (qui est notre cas), on s'authentifie généralement directement avec le compte cible final via Kerberos. On n'a pas besoin de "devenir" quelqu'un d'autre après la connexion, car on s'est déjà connecté en tant qu'utilisateur ayant les permissions nécessaires.

2. become_method = runas

- **Signification** : C'est l'équivalent Windows du `sudo` sur Linux. Au lieu d'utiliser **su** ou **sudo**, Ansible utilise l'API Windows `runas` pour exécuter une commande avec les jetons de sécurité d'un autre utilisateur.
- **Signifier "en tant qu'admin" ?** Oui, dans le principe. Si on avait **become = True**, Ansible tenterait d'exécuter les tâches en utilisant cette méthode pour prendre l'identité spécifiée dans `become_user`.

3. become_user = admin_ansible

- **Signification** : C'est le compte de service (l'identité cible) qu'on souhaite utiliser pour effectuer les actions administratives.
- **Pourquoi admin_ansible et pas un autre ?**
 - **Sécurité (Principe du moindre privilège)** : On évite d'utiliser le compte Administrator local, qui est trop puissant et très ciblé par les malwares.
 - **Traçabilité** : En utilisant un compte spécifique comme admin_ansible, les journaux d'événements Windows (Event Viewer) montrent clairement que les modifications ont été effectuées par ce compte de service dédié, et non par le compte personnel de driss ou un compte générique.
 - **Gestion des permissions** : Ce compte a probablement été configuré spécifiquement dans l'Active Directory pour posséder exactement les droits nécessaires à la gestion IIS et aux tâches qu'on a automatisées (lecture/écriture des sites web, gestion des pools, etc.), sans être "Domain Admin" (ce qui serait dangereux).

Résumé pour ton architecture :

Dans le fichier ansible.cfg, même si become = False, ces paramètres sont importants car ils définissent **le contexte de sécurité prévu** pour l'infrastructure.

Note importante : Si un jour on doit exécuter une tâche qui nécessite des droits plus élevés (par exemple modifier une clé de registre système critique), on devrait passer temporairement **become = True** dans le playbook pour cette tâche précise. le système utilisera alors **runas** pour agir en tant que admin_ansible.